

PROGRAMMATION FONCTIONNELLE 2 : TD6

février 2026 — Pierre Rousselin

On garde dans ce TD les opérations et notations du TD précédent.

Exercice 1 : injection

Dans chacun des cas suivants, donner l'état de la preuve après la tactique `injection` et terminer la preuve.

1. `Lemma ex1_1 (n m : nat) : S (S (S n)) = S (S m) -> S n = m.`
`Proof. intros H. injection H.`
2. `Lemma ex1_2 (n m : nat) : 2 + (S n) = S (S m) -> S n = m.`
`Proof.`
`intros H. injection H.`
3. `Lemma ex1_3 (l1 l2 : list nat) (n : nat) : 1 :: 2 :: l1 = 1 :: n :: l2 ->`
`2 = n /\ l1 = l2.`
`Proof.`
`intros H.`
`injection H as Hn Hl.`
4. `Lemma ex1_4 (l1 l2 : list nat) :`
`[1; 2; 3] ++ l1 = 1 :: 2 :: l2 -> 3 :: l1 = l2.`
`Proof.`
`intros H. injection H.`

--- * ---

Exercice 2 : discriminate

Dans chaque cas suivant, réduire les termes dans l'égalité fournie à `discriminate` ainsi que le ferait Rocq et dire si la tactique `discriminate` donnée réussit ou échoue.

1. `Lemma ex2_1 : 4 <> 2.`
`Proof. intros H. discriminate H.`
2. `Lemma ex2_2 (n : nat) : 2 + n <> 1.`
`Proof.`
`intros H. discriminate H.`
3. `Lemma ex2_3 (n : nat) : S (S n) <> 2.`
`Proof.`
`intros H.`
`discriminate H.`
4. `Lemma ex2_4 (n : nat) : n + 2 <> 1.`
`intros H.`
`discriminate H.`
5. `Lemma ex2_5 (l : list nat) : 42 :: 1 <> [].`
`Proof.`
`intros H.`
`discriminate H.`
6. `Lemma ex2_6 : negb (negb true) <> false.`
`Proof.`
`intros H.`
`discriminate H.`

--- * ---

Exercice 3 : Discriminate pour éliminer les cas impossibles

Écrire la preuve de chacun des lemmes suivants avec une preuve par cas, en utilisant `discriminate` pour éliminer les cas impossibles.

1. `Lemma negb_eq_true (b : bool) : negb b = true -> b = false.`

2. **Lemma** `add_eq_0` $(n\ m : \text{nat}) : n + m = 0 \rightarrow n = 0 \wedge m = 0.$
3. **Lemma** `app_eq_nil` $\{A : \text{Type}\} (l1\ l2 : \text{list } A) :$
 $l1 ++ l2 = [] \rightarrow l1 = [] \wedge l2 = [].$

--- * ---

Exercice 4 : discriminate et induction

On rappelle les définitions des fonctions `filter`, `all` et `any`, vues au TD précédent :

```

Fixpoint filter {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => []
  | h :: q => let fq := filter p q in if p h then h :: fq else fq
  end.
Fixpoint all {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => true
  | h :: q => if p h then all p q else false
  end.
Fixpoint any {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => false
  | h :: q => if p h then true else any p q
  end.

```

1. Prouver :

```

Lemma all_filter {A : Type} (p : A -> bool) (l : list A) :
  all p l = true -> filter p l = l.

```

2. Prouver :

```

Lemma any_filter {A : Type} (p : A -> bool) (l : list A) :
  any p l = true -> filter p l <> [].

```

--- * ---